(Machine) Learning the Barrier from the Beam

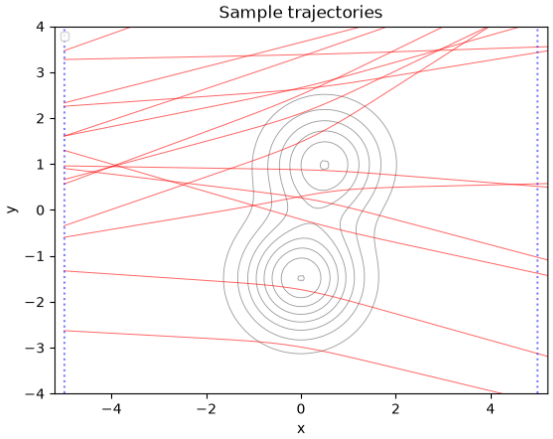
Group project 3; 2026

1 Problem description

We are going to look at a *classical* two-dimensional setup, in which we have particles moving under the influence of a potential. This potential is an a-priori arbitrary function (within some reason) of the $\vec{r} = (x, y)$ coordinates, so can be written as $V(\vec{r})$. The equation of motion for a particle is

$$\frac{\mathrm{d}^2}{\mathrm{d}t^2}\vec{r}(t) = -\nabla V(\vec{r})\,. \tag{1}$$

The particles enter the region of non-vanishing potential from the left at $x = -L$ and exit this region again at $x = +L$.



The question is now as follows: assume that for each particle we know the speed $\vec{v}(x = -L)$ and the position $\vec{r} = (-L, y)$ when it enters this region, and assume we also know the speed and position on exit, so $\vec{v}(x = +L)$ and $\vec{r}(+L, y)$, can we use this to determine $V(\vec{x})$? Clearly just sending a single particle through the potential will not be enough, but the more particles we send through the interacting region (under various angles and with various speeds) the more information we get about the potential. But how can we obtain $V(\vec{x})$ from this set of "initial data" and "final data"? This problem is called an "inverse problem", because we try to figure out the potential given the scattering data (instead of determining the scattering data given the potential).

This problem is very relevant in real-world applications. Think of, for instance, the problem of imaging the internals of the human body using tomography. In this process, one sends particles or waves into the body from one side, and measures what comes out on the other side. The task is then to reconstruct the interior from these data. A similar thing happens in seismic imaging, in which we send seismic waves into the interior of the earth, measure them when they come out again, and want to reconstruct a density profile of the earth's interior.

We will not use such measured data, but instead create our own "measurements". First, we will let someone else pick a potential $V(x, y)$. Then we send particles in from one end, compute their trajectories numerically, and measure how they come out on the other end. We then set ourselves the task of reconstructing the potential if we only know the ingoing and outgoing measurements.

This general "inverse problem" has a long history, backed up by loads of literature, most of which contains a serious amount of maths. Here, we will solve it with a relatively new approach, using machine-learning techniques to make the problem very simple to describe and turn into code. If you manage to do this and have time left over, or if you continue the project into the individual phase, there is also a quantum-mechanical version of this to explore.

2 Ingredients

2.1 Trajectories

A key ingredient of the inverse problem is that we need to be able to compute the trajectory of a particle *given* a potential. This is all governed by the Newton equation (1), but of course that equation may not have an analytic solution if the potential $V(\vec{r})$ is sufficiently complicated. We will therefore resort to numerical techniques. This section contains a brief reminder of how this works.

If you have a differential equation like (1), the logic to turn it into a (system of) finite difference equation(s) is always similar. First, introduce additional variables to turn it into two first order equations:

$$\dot{\vec{r}} = \vec{v}\,, \qquad \dot{\vec{v}} = -\nabla V(\vec{r})\,. \tag{2}$$

In the most naive approach, you would then discretise the time axis, so that $t = n\Delta$ for some small Δ, and rewrite the derivatives as

$$\dot{\vec{r}}_n = \frac{\vec{r}_{n+1} - \vec{r}_n}{\Delta}\,, \qquad \dot{\vec{v}}_n = \frac{\vec{v}_{n+1} - \vec{v}_n}{\Delta}\,, \tag{3}$$

where $\vec{r}_n = \vec{r}(t = n\Delta)$ and ditto for $\vec{v}$. This turns the system above into

$$\vec{r}_{n+1} = \vec{r}_n + \Delta\vec{v}_n\,, \qquad \vec{v}_{n+1} = \vec{v}_n - \Delta\,\nabla V(\vec{r}_n)\,. \tag{4}$$

We can iterate this from the initial data $(\vec{r}_0, \vec{v}_0)$ until some time step where $\vec{r}_m = (L, \ldots)$, and then read off our final data.

The above "Euler method" is known to be numerically unstable, in that errors tend to accumulate and make the numerical solution deviate more and more from the true solution as time progresses. Better discretisation methods are available, and you may have seen some of these in Modelling, Numerical Methods, or Computational Physics.

Exercise (week 1): Lookup the so-called "4th order Runge-Kutta" method, or refresh your memory on this, and write some code to numerically integrate the path of a particle like in the example above (so, fix the initial data to be $x(t = 0) = -L$ and some position $y(t = 0)$, together with initial velocity $\vec{v}(t = 0)$. Use whatever potential you like; the graph above was made with

$$V(x, y) = 2\exp\Big(-\frac{1}{2}(x^2 + (y + 1.5)^2)/(0.8^2)\Big) + 1.5\exp\Big(-\frac{1}{2}((x - 0.5)^2 + (y - 1)^2)/(0.8^2)\Big)\,. \tag{5}$$

Vary the initial data and verify that the paths make sense qualitatively.

2.2 Differentiable physics

Once we know how trajectories behave in a potential, we would like to understand how *changes* to that potential influence the trajectories. A question you could ask, for instance, is how the trajectories depend on the *locations of the centres* of the two Gaussian peaks in the potential (5). You could study that question by generating many potentials of that form, each with different locations of the peaks. But there is a better way to approach this problem.

To understand this, let us consider a *much* simpler example, in which the dynamics is not governed by a full function $V(x, y)$ like above, but by a *single parameter*. An example of this type is the pendulum. Its time evolution is described by the following form of the equation of motion and initial conditions,

$$\ddot{\theta} = -\frac{g}{\ell}\sin\theta\,, \quad \theta(t=0) = \theta_0\,, \quad \dot{\theta}(t=0) = 0\,, \tag{6}$$

where ℓ is the length of the pendulum and g the gravitational constant. Here, the analogue of the problem sketched in the first paragraph above is to understand how the trajectories $\theta(t)$ depend on the parameter g/ℓ. For small θ, we know the answer analytically, because we can expand the sine as $\sin\theta = \theta + \ldots$ to reduce the system to a harmonic oscillator. But for generic values the motion will be non-harmonic.

Suppose now that someone does not tell us the value of g/ℓ, but gives us a set of measurements of the angle θ at times $\{t_i\}$, so

$$(t_1, \theta(t=t_1)),\ (t_2, \theta(t=t_2), \ldots)\,. \tag{7}$$

The question is now how we can use these measurements to determine the value of g/ℓ?

The most brute-force way to do this is to make a random *guess* for g/ℓ, simulate the time evolution $\theta(t)$ for a time range which includes all the measurements, and then determine how well this trajectory fits the data (e.g. by computing the sum of squares of errors for each time t_i). If this error is large, we guess a different g/ℓ, and run again. Hopefully at some point we will hit the jackpot.

More systematic is to determine this error for two values of g/ℓ which are close together, and use this to estimate how the error depends on g/ℓ. To first approximation, this gives us the *derivative* of the error with respect to the parameter which we are varying,

$$\frac{\mathrm{d}E}{\mathrm{d}(g/\ell)} = \lim_{h\to 0}\frac{E(g/\ell + h) - E(g/\ell)}{h}\,. \tag{8}$$

We can then use the value of this derivative to determine whether we have to make g/ℓ larger or smaller in order to get to a better fit of the trajector to the data (see also Newton's root finding algorithm).

Exercise (week 2 and 3): The following partial code uses the JAX library (see section 5.1) to integrate the pendulum equation of motion by starting from $\theta(t=0) = \theta_0$ and $\mathrm{d}\theta/\mathrm{d}t(t=0) = 0$, using Runge-Kutta. The function `final_angle` returns the value of $\theta(t=T)$. Try to understand this code and extend it so that it runs. Then extend it so that you can print the derivative

$$\left.\frac{\mathrm{d}\theta}{\mathrm{d}(g/\ell)}\right|_{t=T} \approx \frac{\theta(t=T+h) - \theta(t=T)}{h}\,. \tag{9}$$

Can you use this information about the derivative to find $g\ell$ which makes the system hit a partial final angle $\theta(t = T) = \theta_f$?

```
g = 9.81
T = 5.0
N_STEPS = 500
DT = T / N_STEPS

def rhs(z, ell):
    theta, omega = z
    return jnp.array([omega, -(g / ell) * jnp.sin(theta)])

def rk4_step(z, ell):
    k1 = rhs(z, ell)
    k2 = rhs(z + 0.5 * DT * k1, ell)
    k3 = rhs(z + 0.5 * DT * k2, ell)
    k4 = rhs(z + DT * k3, ell)
    return z + (DT / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)

def trajectory(ell, theta0):
    z0 = jnp.array([theta0, 0.0])

    def step(z, _):
        z_new = rk4_step(z, ell)
        return z_new, z_new[0]      # (carry, stored output)

    _, thetas = jax.lax.scan(step, z0, None, length=N_STEPS)
    return jnp.concatenate([jnp.array([theta0]), thetas])

def final_angle(ell, theta0):
    return trajectory(ell, theta0)[-1]
```

But we can do better than this. The above uses a discretised form of the derivative. But *in principle*, all operations and functions in the code above have analytic derivatives. So you could write a function

```
def drhs(z, ell):
    theta, omega = z
    return jnp.array([omega, -jnp.sin(theta)])
```

which returns the *derivative* of the rhs of the equation of motion with respect to g/ℓ, evaluated at the given values of $z = (\theta, \dot{\theta})$ and g/ℓ. And then you could similarly write a function `drk4_step(z, ell)` to give the derivative of the RK4 step with respect to g/ℓ, using the `drhs` above. And if you push that idea all the way through all functions, you can write a `dfinal_angle` function which returns exactly the derivative of the final angle with respect to g/ℓ, evaluated at a particular value g/ℓ and θ_0.

This is just about doable by hand for small code, but always error-prone, and becomes infeasible for larger code. But it's the principle that matters: every operation that goes into the computation

of the final result is such that we can also compute the *derivative* of that operation with respect to g/ℓ. Fortunately, this process can be automated, and that's precisely what the JAX algorithm `grad` does for us. This is what is called "differentiable physics": even though we have a numerical simulator, we can write down functions that compute the derivative of the final result with respect to the parameter(s) in the problem. This derivative depends on the numerical errors made in the discrisation of the Runge-Kutta solver, but does *not* suffer from errors caused by computing the derivative as a finite difference.

Exercise (week 2): Study the information about the JAX `grad` operator in section 5.1 as well as in online JAX resources. Then use this to implement the gradient (9) and compare with the finite-difference you found before.

Exercise (week 2 or 3): Now go back to the problem in which you are given a set of data points generated from a particular pendulum, and your task is to deduce g/ℓ. You want to use (8), so you need to write down what the error is, by e.g. computing the sum of squares between what your simulation predicts and what the data says.

2.3 Machine learning

Neural networks are, at their very core, nothing but fancy data interpolators/extrapolators. In this section we will look at an example to illustrate this, at a level where you can follow things "by hand". It is also a good example to show you how to get started with JAX.

The idea is as follows. Suppose someone gives you data points (x_n, y_n) for $n \in \{0, \ldots, N\}$, which are sampled from a particular function, e.g. $\sin(x)$ (perhaps with some noise added). We want to construct a neural network which can be trained to reproduce the function value for arbitrary values of x. That is to say, we want that if we feed it a number x, it will produce for us the number $\sin(x)$. There are many ways in which you can construct such interpolating black boxes, e.g. using linear interpolation, or using splines, or various other ideas.

We will use a very simple neural network intead. It consists of a single "input node" (the value x) and a single "output node" (the prediction for the value y), and one layer of internal nodes. The values of all nodes (except the input nodes) are computed by using a "weight matrix" to obtain the next layer nodes $x_i^{(L+1)}$ from the previous layer nodes $x_i^{(L)}$, as in

$$x_i^{(L+1)} = \sigma\big(W_{ij}^{(L)} x_j^{(L)} + b_i \big) \,, \tag{10}$$

where summation over j is implied. The function σ is the "activation function" of the nodes, and is some non-linear monotonic function, e.g.

$$\sigma(x) = \frac{1}{1+e^{-x}} \,. \tag{11}$$

This is repeated layer-by-layer, until we reach the end. The neural network is thus nothing but a complicated non-linear function of the inputs.

With random matrices $W_{ij}^{(L)}$, the output is of course not going to be anywhere near the values of the $\sin(x)$ function that we are after. In order to determine how far we are off, use a *loss function*,

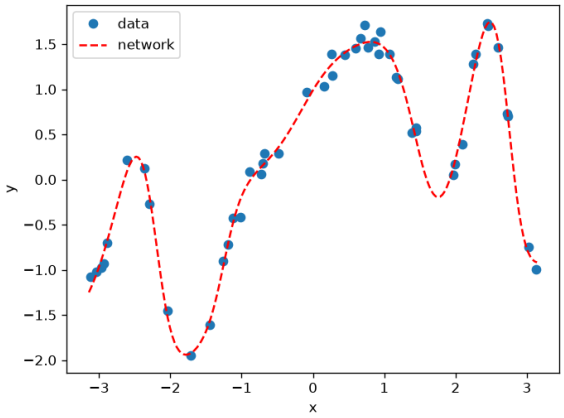
$$\text{loss} = \sum_n |\text{net}(x_n) - y_n|^2 \,, \tag{12}$$

where the data pairs (x_n, y_n) are the data points we want to interpolate. We can ask ourselves how this loss depends on all the weight matrix elements, so what is

$$\frac{\partial \text{loss}}{\partial W^l_{ij}} \,. \tag{13}$$

If we know these derivatives (or gradients) or the loss function with respect to the weights, we can change the weights slightly so that the loss becomes less, similar to what we did with the pendulum.

Exercise (week 4): Plan and create a program to generate a set of data points from a known function, and then uses a neural network to fit a smooth curve through those points. Something like in the figure below (which was made with one hidden layer with 16 nodes, for a total of 32 weights).



Below is a roadmap for writing such a fitting program in JAX. Do the steps in order, and *test each one* before moving on (printing the shapes of arrays with `.shape` catches most mistakes).

1. **Data.** Pick a function, e.g. $f(x) = \sin x + \cos(x^2)$ as in the example above, and generate $N \approx 50$ random points $x_n \in [-\pi, \pi]$ with $y_n = f(x_n) + \epsilon_n$, where ϵ_n is Gaussian noise of size ~ 0.1. Use

   ```
   rng = np.random.default_rng(seed)
   ```

   with a fixed seed (so your runs are reproducible), and convert to JAX arrays with `jnp.asarray`.

2. **Parameters.** Store the weights as a Python list of pairs `(W, b)`, one pair per layer, as in `init_params` in section 5.1. For `LAYERS = [1, 16, 1]`, what are the shapes of the two `W`'s and `b`'s? Why should the initial `W` be random rather than zero? Note that JAX random numbers need an explicit "key" (`jax.random.PRNGKey(seed)`), which you `jax.random.split` every time you need fresh numbers.

3. **Forward pass.** Write `mlp(params, inp)`, which takes a vector of input-node values and, for every layer, computes `inp = jnp.tanh(inp @ W + b)` (here $\sigma = \tanh$; the sigmoid above works too). Do *not* apply σ in the last layer: why not? A plain Python `for` loop over the layers is fine here. Then write `predict_one(x, params)`, which turns a single number `x` into a length-1 array `jnp.array([x])`, feeds it to `mlp`, and returns the single output number.

4. **Many points at once.** Rather than looping over the data, let JAX "vectorise" your function:

   ```
   predict = jax.vmap(predict_one, in_axes=(0, None))
   ```

   The `in_axes` say: map over the first argument (an array of x-values), but pass the second (`params`) unchanged to every call. Check that `predict(x_data, params)` has the same shape as `y_data`.

5. **Loss.** Write `loss(params)`, returning the mean of `(predict(x_data, params) - y_data)**2` (the mean instead of the sum only rescales the loss by $1/N$). It takes *only* the parameters as an argument, because those are what we differentiate with respect to; the data can be global variables.

6. **Gradients.** `loss_and_grad = jax.value_and_grad(loss)` gives a new function; the expression `loss_and_grad(params)` returns the loss together with its derivatives with respect to *all* weights. The derivatives come packed in exactly the same structure as `params`: a list of `(dW, db)` pairs with matching shapes. Verify this by printing the shapes.

7. **Optimiser.** You could now update every weight by hand as $W \to W - \eta\, \partial\,\text{loss}/\partial W$ for some small "learning rate" η, but the `optax` library does this for you, with a smarter update rule called "Adam". Create the optimiser and its internal state once, before training:

   ```
   opt = optax.adam(LEARNING_RATE)       # e.g. 1e-2
   opt_state = opt.init(params)
   ```

8. **Training step.** Write a function `step(params, opt_state)` which calls `loss_and_grad`, and then updates the weights with

   ```
   updates, opt_state = opt.update(grads, opt_state)
   params = optax.apply_updates(params, updates)
   ```

   returning `params, opt_state, loss`. Put `@jax.jit` on the line above `def step`: this compiles the function on its first call, making all later calls much faster. Then call `step` a few thousand times in a Python `for` loop, printing the loss every 200 iterations. Play with the learning rate: what happens if it is too large or too small?

9. **Plot.** Evaluate `predict` on a fine grid `jnp.linspace(-np.pi, np.pi, 200)` and plot it together with the data points. Also store the loss at every iteration in a list, and plot it (on a logarithmic scale) against the iteration number.

10. What happens if you increase the number of layers or nodes in each hidden layer? Study the loss function as a function of the number of iterations, and try to understand concepts like

overfitting.

3 The inverse problem

3.1 Neural network

Our problem is of a somewhat different nature than the story above. We will use a neural network which takes x, y as inputs, and for any value pair spits out $V(x, y)$. The goal is to train the weights in such a way that, when we send particles through this potential, we reproduce the ingoing/outgoing data of the "measured" particle trajectories.

Exercise (week 5 and 6): Plan and build a program that integrates trajectories through a potential which is generated as the output of a neural network with two inputs (the position (x, y)) and one output $V(x, y)$. It should contain a function which takes the input data for a trajectory and produces the output data. You should construct this in such a way that you can take the `grad` of the error function,

Below is a roadmap for such a program. Much of it re-uses what you built before: the RK4 integrator (week 1), `jax.grad` and `jax.lax.scan` (week 2) and the network and training loop of section 2.3. As before, *check every step* before moving on.

1. **Setup.** Use $L = 5$, an initial forward velocity $v_0 = v_x(t = 0) = 3$, and integrate for a fixed time $T = 4.5$ in `N_STEPS = 80` RK4 steps. How long does a particle take to cross from $x = -L$ to $x = +L$ if $V = 0$? Why should T be somewhat larger than that?

2. **True potential and forces.** Write `V_true(x, y)` as in (5), and get the force components with `jax.grad(V_true, argnums=0)` (for $\partial_x V$) and `argnums=1` (for $\partial_y V$). *Check:* compare with the derivative you compute by hand at one or two points.

3. **Integrator.** Write `rk4_step(state, rhs_fn)` with `state` $= (x, y, v_x, v_y)$, where `rhs_fn(state)` returns the time derivative of the state. Passing the right-hand side as an argument lets you use the *same* integrator for the true potential and, later, for the network. Then write `integrate_traj(y0, vy0, rhs_fn)` which uses `jax.lax.scan` to return the whole trajectory, an array of shape `(N_STEPS+1, 4)`. *Checks:* with $V = 0$ you should get straight lines; with `V_true`, the energy $\frac{1}{2}\vec{v}^2 + V$ should be (nearly) constant along the trajectory.

4. **Exit data.** The particle does not land exactly on $x = L$ at one of the time steps. Write `exit_state_from_traj(traj)` which finds the first time step with $x \geq L$ (hint: `jnp.argmax` of a boolean array gives the index of the first `True`) and linearly interpolates y and v_y between that step and the previous one. *Check:* for $V = 0$, what should y_{exit} and $v_{y,\text{exit}}$ be, given y_0 and $v_{y,0}$?

5. **Observations.** Pick 100 particles with random $y_0 \in [-3.5, 3.5]$ and random $v_{y,0} \in [-1.2, 1.2]$, and compute their exit data with `jax.vmap` of a function `true_exit(y0, vy0)`. Add a little Gaussian noise to the exit data (e.g. 1% of their range); call the noise sizes `noise_y` and

`noise_vy`. *Check:* plot a dozen trajectories on top of a contour plot of V, as in the figure at the start of these notes. Think about why we vary $v_{y,0}$ as well, and not just y_0.

6. **Network potential.** Use the `init_params` and `mlp` from before, now with a setting of `LAYERS = [2, 24, 24, 1]`, and write

```
def V_net(p_V, x, y):
    inp = jnp.stack([x / L, y / L])
    raw = mlp(p_V, inp)[0]
    return jax.nn.softplus(raw - 2.0)
```

   Why do we divide by L? Look up the `softplus` function: what does it guarantee about V, and what does the -2 do to the potential at the start of training? Get the forces with `jax.grad(V_net, argnums=1)` and `argnums=2`.

7. **Predictions.** Write `predict_exit(y0, vy0, p_V)`, which builds a right-hand side from the network forces, integrates, and returns the exit data. Vectorise it over particles with `jax.vmap(predict_exit, in_axes=(0, 0, None))`. *Check:* the output shapes match those of your observations.

8. **Loss.** Write `loss_fn(p_V)` as the mean squared error in y_{exit} divided by `noise_y**2`, plus the same for $v_{y,\text{exit}}$, times $\frac{1}{2}$. Why divide by the noise squared? What value of the loss would mean "the network fits the data as well as the noise allows"? Take `jax.value_and_grad(loss_fn)` as before; note that the gradient now flows through the entire RK4 integration, for all particles at once. This is the differentiable physics of week 2 and 3, with a whole network of weights instead of the single parameter g/ℓ.

9. **Optimiser.** This problem is harder than the curve fit, so we use two extra tricks from `optax`: a learning rate which decays from 10^{-2} to 10^{-4} during training (`optax.exponential_decay`), and clipping of overly large gradients (`optax.clip_by_global_norm(0.1)`). Combine them as `optax.chain(clip, optax.adam(schedule))`; see the `optax` documentation for the arguments.

10. **Training.** Use the same jitted `step` function and loop as in section 2.3, for about 600 iterations, storing the loss. The first call will take a while (JAX is compiling); later calls should be fast. *Check:* plot the loss on a log scale; it should approach the value you found in the previous step.

11. **Diagnostics.** Evaluate `V_true` and the trained `V_net` on a 100×100 grid (use `jnp.meshgrid` and a nested `jax.vmap(jax.vmap(...))`) and plot them next to each other with `plt.pcolormesh`, together with their difference. Also scatter-plot predicted against observed exit data (the points should lie on the diagonal), and overlay true and predicted trajectories. Where is the reconstruction good, and where is it bad? Can you explain why?

12. **Experiments.** What happens if you send all particles in horizontally ($v_{y,0} = 0$)? With fewer particles, or more noise? Try a different potential.

4 Week-by-week plan

Week 1* Problem setting, numerical methods, literature inventory.

Week 2* Differentiable physics, pendulum example.

Week 3 Read about neural networks, build and run a one.

Week 4* Scattering against potential: design.

Week 5 Scattering against potential: implementation.

Week 6* Scattering against potential; extensions, other potentials, limitations.

Week 7* No new material; catch up with older material, prepare presentations.

Week 8 Group presentations

Week 9* Mock oral exam

Week 10 Oral exam

We meet in weeks marked with a star.

5 Coding tools and libraries

5.1 JAX

In machine learning, the goal is almost always to let some black box (the neural network) compute a function $L(x, \{w_i\})$ (the "loss", or "cost"). The black box depends on a (large) set of parameters, the weights $\{w_i\}$. We then want to use the derivatives with respect to the weights to determine how we need to change the weights so that the loss decreases. The neural network is, however, nothing but a very complicated nonlinear function of x and the $\{w_i\}$, so what we need is some tool to evaluate derivatives of these complicated functions with respect to the w_i, evaluated at arbitrary values of the inputs x. JAX is a Python library which provides precisely this (there are many others, PyTorch is another well-known one).

To see how this works, let's look at an example. In the following code, we define a function $f(x) = \cos(x^2) + \sin(x)$, and then use JAX to obtain a new python function $g(x)$ which, for every input x, computes the derivative $f'(x)$.

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt

def f(x):
    return jnp.cos(x**2) + jnp.sin(x)

x = jnp.linspace(0.0, jnp.pi, 100)
```

```
g = jax.vmap(jax.grad(f))

plt.plot(x, f(x))
plt.plot(x, g(x))
plt.show()
```

Under the hood, JAX "traces" the input x through the computation, and uses the fact that it knows the derivative of all elementary functions which appear in this computation, together with the chain rule, to construct a program that can compute the derivative. This is called "automatic differentiation"; for more information see e.g. ... Note that this program *never* uses finite differences to compute the derivatives.

It often happens that you need to create loops, for instance to evaluate a function for many values of the time parameter t. In JAX, you need to avoid doing 'for i in range(...)' type Python loops, as those are very slow. The basic building block for loops in JAX is `jax.lax.scan`. It works as follows:

```
import jax
import jax.numpy as jnp

def fun(state, x):
    newstate = ...
    y = ...
    return newstate, y  # new state, output

initial = ...
values  = jnp.array(...)
state, output = jax.lax.scan(fun, initial, values)
```

This calls `fun` once for every value in `values`, passing it the current state. The function computes a value which should go into the output array, and updates the state. Here is an example which evaluates `jnp.sin` for each value in a list:

```
def fun(state, x):
    return state, jnp.array([x, jnp.sin(x)])

state, output = jax.lax.scan(fun, 0, jnp.linspace(0, jnp.pi, 10))
print(output)
```

This does not use or update the `state` at all. You could do the same thing by letting the state take care of the x-value update, something like

```
def fun(state, x):
    return state+jnp.pi/9, jnp.array([state, jnp.sin(state)])

state, output = jax.lax.scan(fun, 0, None, length=10)
print(output)
```

This example, in contrast, does not use the `values` at all, but instead constructs the x-value from the state. If you find this difficult to wrap your head around, there is also `jax.lax.fori_loop`. See the JAX documentation for more information.

In JAX, you construct neural networks by simply writing out how the inputs get multiplied with weight matrices at every layer.

```
import jax
import jax.numpy as jnp

def init_params(sizes):
    out = []
    key = jax.random.PRNGKey(SEED)
    for i in range(len(sizes) - 1):
        key, sub = jax.random.split(key)
        scale = jnp.sqrt(1.0 / sizes[i])
        W = jax.random.normal(sub, (sizes[i], sizes[i + 1])) * scale
        b = jnp.zeros(sizes[i + 1])
        out.append((W, b))
    return out

LAYERS = [1, 5, 1]
params = init_params(LAYERS)
opt_state = opt.init(params)
n_p = sum(W.size + b.size for W, b in p_V)
```

References

[1] 3Blue1Brown, "Deep learning". An easy-listening introduction if you have had no previous exposure to neural networks. The first 4 chapters are relevant for this project.

[2] Higham & Higham, "Deep Learning: An Introduction for Applied Mathematicians", SIAM Review 61(4), 2019.

[3] Fleuret, "The Little Book of Deep Learning", 2023.

[4] The JAX authors, "JAX 101".

[5] N. Thuerey et al., "Physics-based Deep Learning", 2021